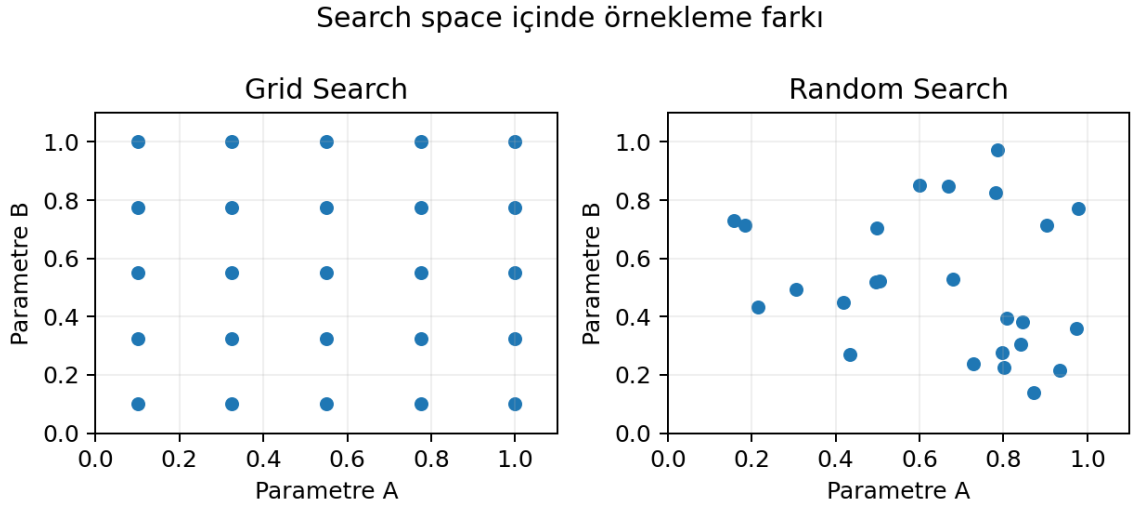
22. HYPERPARAMETER OPTIMIZATION

GridSearchCV • RandomizedSearchCV • Optuna • Search Space • Early Stopping



Bu modülün amacı
Hyperparameter tuning'i rastgele parametre denemek olarak değil; doğru objective, validation tasarımı, search space, hesaplama bütçesi ve overfitting kontrolüyle yürütülen sistematik bir optimizasyon problemi olarak öğrenmek.

Kazanım	Modül sonunda
Temel ayrım	Model parameter ile hyperparameter farkını net açıklayabilirsin.
Search yöntemleri	Grid, random ve Bayesian/Optuna yaklaşımını karşılaştırabilirsin.
Search space	Parametre aralıklarını lineer/log/categorical olarak doğru tasarlayabilirsin.
Validation	Tuning sırasında leakage ve validation overfitting riskini yönetebilirsin.
Efficiency	Pruning, early stopping ve parallelization ile hesaplama bütçesini verimli kullanabilirsin.
Reporting	Best trial kadar search distribution ve stability bilgisini raporlayabilirsin.

1. Hyperparameter nedir?

Model parameter veriden öğrenilir; hyperparameter ise öğrenme sürecinin yapısını veya model kapasitesini kontrol eder. Hyperparameter optimization, validation performansını kullanarak bu ayarların iyi kombinasyonlarını arar.

Parameter	Hyperparameter
Linear regression coefficient	Ridge alpha
Tree split thresholds	max_depth / min_samples_leaf
Neural network weights	learning_rate / batch_size
Boosting tree leaf values	n_estimators / learning_rate / depth

Ana fikir
Hyperparameter tuning eğitim verisine değil validation performansına göre karar verir. Bu yüzden tuning, model training kadar evaluation tasarımına da bağlıdır.

2. Neden tuning yapıyoruz?

- Underfitting ve overfitting arasındaki kapasite dengesini ayarlamak.
- Optimization davranışını iyileştirmek.
- Regularization gücünü seçmek.
- Modelin veri yapısına uygun karmaşıklığını bulmak.
- Aynı model ailesinde iyi operating region bulmak.

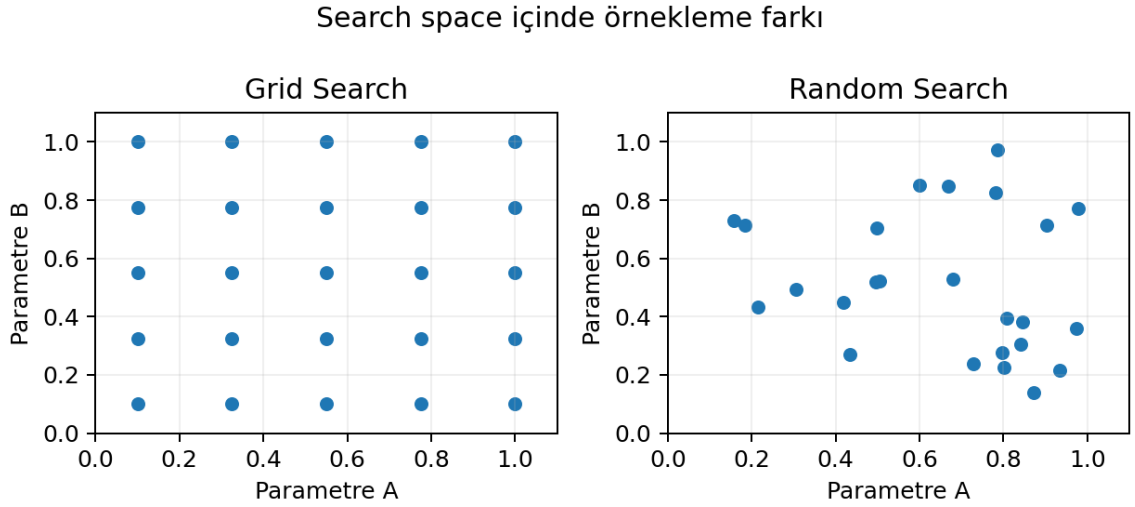
Tuning her şeyi çözmez
Kötü target, leakage, yanlış split veya zayıf feature set hyperparameter search ile düzelmez.

3. Tuning workflow

1. Business objective ve primary metric belirle.
2. Train/validation/test veya CV stratejisini kilitle.
3. Baseline ve default model kur.
4. Search space tanımla.
5. Search algorithm ve trial budget seç.
6. Trial sonuçlarını kaydet.
7. Best region ve stability analiz et.
8. Final configuration ile retrain et.
9. Test set üzerinde yalnızca bir final değerlendirme yap.

4. Grid Search

Grid Search önceden belirlenen tüm hyperparameter kombinasyonlarını dener. Küçük ve iyi sınırlandırılmış search space için anlaşılır ve reproducible bir yöntemdir.




```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__max_depth": [3, 5, 8],
    "model__min_samples_leaf": [1, 5, 20]
}

search = GridSearchCV(
    pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
search.fit(X_train, y_train)
```

Grid dezavantajı
Boyut sayısı arttıkça kombinasyon sayısı çarpılır. 6 parametrenin her birinde 5 seçenek varsa 15,625 kombinasyon gerekir.

5. Randomized Search

Randomized Search her parametre için distribution tanımlar ve belirli sayıda kombinasyonu örnekler. Büyük search space içinde daha verimli coverage sağlar.

```
from scipy.stats import loguniform, randint
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "model__learning_rate": loguniform(1e-3, 0.3),
    "model__max_depth": randint(2, 10),
    "model__subsample": [0.6, 0.8, 1.0]
}

search = RandomizedSearchCV(
    pipeline,
    param_distributions=param_dist,
    n_iter=50,
```

```
    cv=5,
    scoring="average_precision",
    random_state=42,
    n_jobs=-1
)
```

Neden random?
Çoğu problemde performansı birkaç hyperparameter güçlü biçimde etkiler. Random search bütçeyi gereksiz grid eksenlerine harcamaz.

6. Grid vs Random

Özellik	Grid Search	Random Search
Coverage	Sabit grid noktaları	Dağılımdan örnekler
Büyük space	Pahalı	Daha verimli
Reproducibility	Yüksek	random_state ile yüksek
Ön bilgi	Aralıkları çok iyi bilmek faydalı	Geniş aralık keşfi için güçlü
Budget kontrolü	Kombinasyon sayısı ile	n_iter ile doğrudan

Pratik strateji
Önce geniş Random Search, sonra iyi bölgeyi daraltıp daha odaklı search yapmak sık kullanılan güçlü bir yaklaşımdır.

7. Search space tasarımı

Optimization kalitesini search algoritmasından önce search space belirler. Yanlış aralık seçersen optimizer iyi config bulamaz.

Parametre tipi	Örnek dağılım
Scale-sensitive	learning_rate, alpha -> log scale
Integer	depth, min_samples_leaf -> randint
Probability/rate	subsample -> uniform bounded
Categorical	booster, criterion -> list
Conditional	booster seçimine göre alt parametreler

Log scale
1e-4 ile 1e-3 arasındaki fark, 0.100 ile 0.101 arasından daha anlamlı olabilir. Learning rate ve regularization parametreleri için loguniform çoğu zaman daha mantıklıdır.

8. Objective metric seçimi

Problem	Primary tuning metric
Regression, büyük hata pahalı	RMSE / custom cost
Robust regression	MAE
Balanced classification	ROC-AUC / log-loss
Rare positive	PR-AUC / recall@precision
Capacity-limited outreach	Precision@k / custom value

Probability risk score	Log-loss / Brier + calibration

Tuning hedefi business hedefiyle uyumlu olmalı
Yanlış metric optimize edilirse model teknik olarak iyileşirken business değeri düşebilir.

9. Cross-validation ile tuning

Her hyperparameter configuration tek holdout yerine CV fold ortalamasıyla değerlendirilirse daha stabil seçim yapılır.

```
search = RandomizedSearchCV(
    model,
    param_distributions=space,
    n_iter=60,
    cv=StratifiedKFold(5, shuffle=True, random_state=42),
    scoring="average_precision"
)
```

Ama maliyet artar
60 trial x 5 fold = 300 model fit. Search budget hesaplanırken CV multiplier unutulmamalıdır.

10. Group ve time-aware tuning

Veri yapısı	CV
IID classification	StratifiedKFold
Tekrarlı müşteri/hasta	GroupKFold / StratifiedGroupKFold
Forecast / temporal prediction	TimeSeriesSplit / temporal folds
Event overlap	Purged/embargo yaklaşımı düşünülebilir

Kural
Tuning CV tasarımı deployment gerçeğini taklit etmiyorsa best params sahte olabilir.

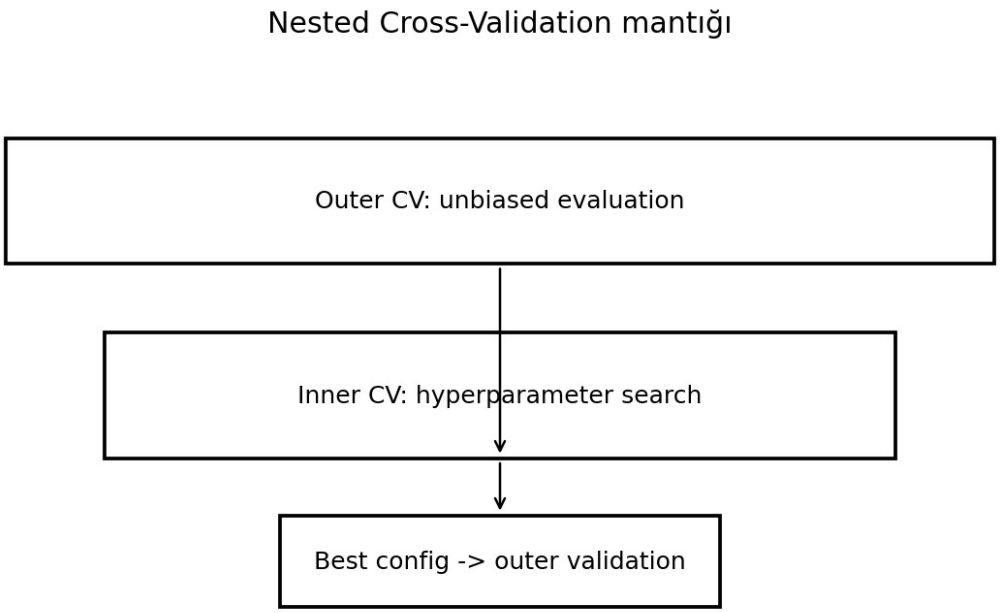
11. Validation overfitting

Çok sayıda trial denedikçe validation setin noise özelliklerine uyum sağlayan config seçme olasılığı artar. Hyperparameter search de overfit olabilir.

- Çok büyük search budget + küçük validation set risklidir.
- Tek split üzerinde yüzlerce trial risklidir.
- Search sonrası aynı validation üzerinde tekrar manuel ayar yapmak riski artırır.

Korunma
Cross-validation, nested CV, ayrı test set ve search space disiplinli sınırlandırma validation overfitting riskini azaltır.

12. Nested Cross-Validation



Nested CV dış foldlarda unbiased evaluation, iç foldlarda hyperparameter selection yapar. Özellikle küçük veri setlerinde model ailesi + tuning performansını daha dürüst tahmin eder.

Ne zaman?
Araştırma, benchmark, küçük/orta dataset ve model farkları küçük olduğunda güçlü yöntemdir; production büyük veride maliyetli olabilir.

13. Early stopping

Boosting ve neural network gibi iterative modellerde validation metric iyileşmesi durduğunda training erken kesilebilir. Bu hem regularization hem compute tasarrufu sağlar.

```
# Kavramsal
model.fit(
    X_train, y_train,
    eval_set=[(X_valid, y_valid)],
    early_stopping_rounds=50
)
```

Search etkisi
n_estimators için dev grid aramak yerine yüksek üst sınır + early stopping daha verimli olabilir.

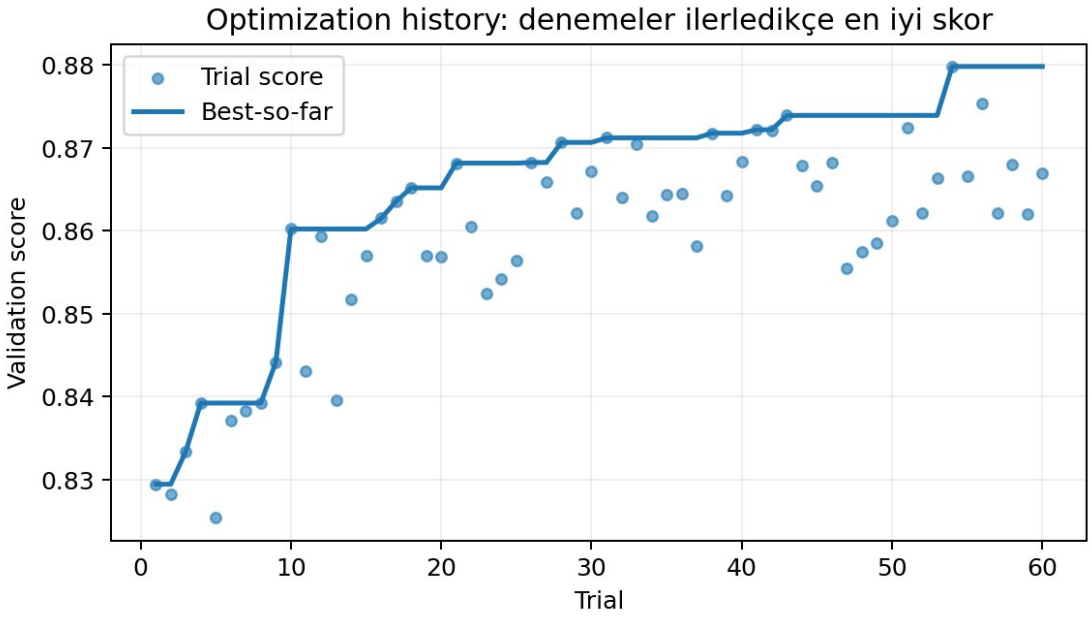
14. Pruning

Optuna gibi frameworklerde kötü giden triallar erken kesilebilir. Böylece hesaplama bütçesi umut verici configurationlara ayrılır.

Pruning vs early stopping
Early stopping tek model trainingini durdurur. Pruning tüm hyperparameter trialını erken sonlandırır.

15. Optuna mantığı

Optuna trial bazlı search frameworküdür. Search space kod içinde tanımlanır, trial objective döndürür ve sampler geçmiş sonuçlara göre gelecek trialları yönlendirebilir.




```
import optuna

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float(
            "learning_rate", 1e-3, 0.3, log=True
        ),
        "max_depth": trial.suggest_int("max_depth", 2, 10),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0)
    }

    score = cross_val_score(
        build_model(params),
        X_train, y_train,
        cv=cv,
        scoring="average_precision"
    ).mean()

    return score

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=80)
```

Avantaj
Conditional search space, pruning, persistent study ve optimization history gibi özellikler production-grade tuning workflowunu kolaylaştırır.

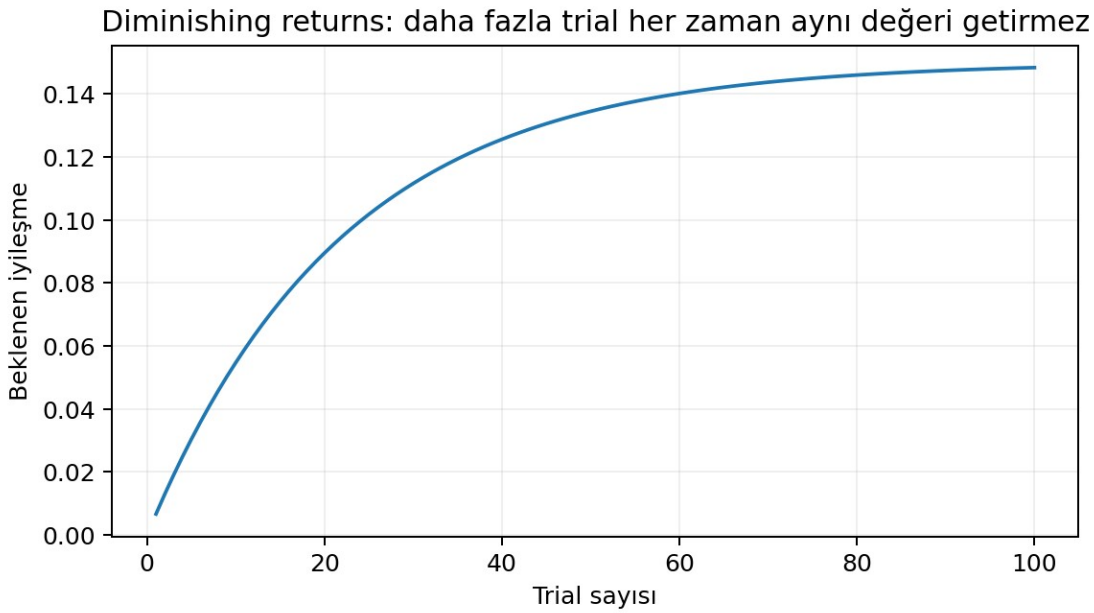
16. Bayesian optimization sezgisi

Random search geçmiş triallardan öğrenmez. Bayesian yaklaşım geçmiş configuration-score ilişkisini kullanarak umut verici bölgeleri daha sık denerken keşfi de sürdürmeye çalışır.

Kavram	Sezgi
Exploration	Az denenmiş bölgeleri araştır
Exploitation	İyi görünen bölgeyi derinleştir
Surrogate / sampler	Gerçek pahalı objective yerine geçmişten yönlendirme
Acquisition	Bir sonraki denemenin faydasını dengele

Optuna notu
Sampler seçimine göre yöntem değişebilir; TPE Bayesian optimization ailesinde yaygın pratik yaklaşımlardan biridir.

17. Compute budget



Tuning sonsuz search değildir. Trial sayısı, CV fold, dataset boyutu ve model training süresi birlikte bütçeyi belirler.

```
total_fits = n_trials * n_cv_folds
approx_hours = total_fits * avg_fit_minutes / 60
```

Diminishing returns
İlk 20-40 trial büyük kazanç sağlarken sonraki 200 trial çok küçük iyileşme getirebilir. Search maliyetini model değerine göre sınırla.

18. Parallelization

- GridSearchCV/RandomizedSearchCV: n_jobs.
- Optuna: parallel workers / shared storage.
- GPU models: trial-level GPU allocation planı gerekir.
- Çok fazla parallel trial memory bottleneck yaratabilir.

Reproducibility
Parallel execution bazı algoritmalarda floating-point/order farkları oluşturabilir. Seed ayarla ama tam determinism garantisini varsayma.

19. Random seed

Search ve model randomness farklı kaynaklardır. Search random_state, CV splitter random_state ve model random_state ayrı kontrol edilmelidir.

```
SEED = 42
cv = StratifiedKFold(5, shuffle=True, random_state=SEED)
model = RandomForestClassifier(random_state=SEED)
```

20. Multi-metric tuning

Birden fazla metric raporlanabilir; fakat hangi metric ile best model seçileceği önceden belirlenmelidir.

```
scoring = {
    "roc_auc": "roc_auc",
    "pr_auc": "average_precision",
    "f1": "f1"
}

search = GridSearchCV(
    model,
    param_grid,
    scoring=scoring,
    refit="pr_auc",
    cv=5
)
```

Pareto düşüncesi
Bazı configurationlar precision artırırken recall azaltabilir. Tek metric yerine business trade-off tablosu da gerekir.

21. Conditional search space

Bazı parametreler yalnız başka bir seçime bağlıdır. Örneğin booster=gbtree seçilirse max_depth anlamlı, linear booster için anlamsız olabilir.

```
booster = trial.suggest_categorical(
    "booster", ["gbtree", "dart"]
)
```

```
    if booster == "dart":
        rate_drop = trial.suggest_float(
            "rate_drop", 0.0, 0.5
        )
```

Grid zorluğu
Conditional space grid ile gereksiz/invalid kombinasyonlar üretirken Optuna benzeri programatik search kolaylaştırır.

22. Parameter importance

Search tamamlandığında sadece best params değil, hangi hyperparameterların performansı daha çok etkilediği de analiz edilebilir.

- Trial-score scatter plot.
- Parameter importance.
- Slice plot.
- Parallel coordinate plot.

Amaç
Bir sonraki search space'i daraltmak ve model davranışını anlamak.

23. Search result analysis

Bakılacak	Neden
Best score	En iyi observed validation
Top 10 configs	Tek bir şanslı trial mı?
CV std	Stability
Train vs valid gap	Overfitting
Parameter region	İyi bölge geniş mi dar mı?
Runtime	Performance/compute trade-off

Sağlam seçim
Best score 0.9001 ama çevresi 0.86 ise; 0.898 alan geniş/stabil bölge daha güvenilir olabilir.

24. Hyperparameter tuning ile feature engineering ilişkisi

Feature pipeline parametreleri de tuning space'e dahil olabilir: imputer strategy, scaler seçimi, feature selection threshold veya polynomial degree.

```
param_grid = {
    "prep__num__imputer__strategy": ["mean", "median"],
    "model__alpha": [0.01, 0.1, 1, 10]
}
```

Leakage-safe
Preprocessing/tuning tek pipeline içinde CV foldlarına göre fit edilmelidir.

25. Tuning ve class imbalance

Class weight, scale_pos_weight, threshold ve resampling stratejisi birbirinden farklı knobs olabilir. Bunları aynı anda optimize etmek search space'i büyütür.

Öneri
Önce doğru metric ve split; sonra model class weighting; threshold tuning ise probability model seçimi sonrasında ayrı validation adımı olarak ele alınabilir.

26. Tuning ve threshold farklı şeylerdir

Hyperparameter tuning	Threshold tuning
Modelin öğrenme davranışını değiştirir	Trained score'u aksiyona çevirir
CV ile model/config seçer	Validation ile operating point seçer
AUC/logloss vb optimize edebilir	Precision/recall/cost trade-off optimize eder

Sıra
Önce model hyperparameterlarını seç, sonra final validation üzerinde threshold/operating policy seç.

27. Regression tuning örneği - Ridge

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

params = {
   "model__alpha": np.logspace(-4, 4, 40)
}

search = GridSearchCV(
   pipeline, params,
   scoring="neg_root_mean_squared_error",
   cv=5
)
```

Neden logspace?
Regularization strength genellikle order-of-magnitude değişimlerine duyarlıdır.

28. Random Forest tuning

Parametre	Etkisi
n_estimators	Daha çok ağaç = daha stabil, daha pahalı
max_depth	Kapasite/overfit

min_samples_leaf	Leaf smoothing
max_features	Tree diversity
class_weight	Imbalance cost

Öncelik
n_estimators çoğu zaman yeterince yüksek tutulur; depth/leaf/features genellikle daha etkili tuning alanıdır.

29. Gradient Boosting / XGBoost tuning

Parametre	Ana trade-off
learning_rate	Küçük = daha yavaş ama hassas
n_estimators	Boosting adım sayısı
max_depth / num_leaves	Tree complexity
subsample	Variance/regularization
colsample	Feature subsampling
reg_alpha/reg_lambda	Regularization

Coupled params
learning_rate ve n_estimators birlikte düşünülmelidir. Düşük learning rate genellikle daha fazla boosting round gerektirir.

30. LightGBM tuning notları

- num_leaves güçlü capacity parametresidir.
- min_data_in_leaf overfitting kontrolünde önemlidir.
- feature_fraction ve bagging_fraction regularization sağlar.
- learning_rate + num_iterations birlikte tune edilir.

Tree growth
LightGBM leaf-wise growth kullanabildiği için num_leaves ve min_data_in_leaf dengesi kritiktir.

31. CatBoost tuning notları

- depth, learning_rate, iterations temel parametreler.
- l2_leaf_reg regularization sağlar.
- Categorical handling sayesinde encoding pipeline sadeleşebilir.
- Overfitting detector / early stopping kullanılabilir.

32. Neural network tuning

Hyperparameter	Örnek
Learning rate	En kritiklerden biri
Batch size	Gradient noise / memory
Depth/width	Capacity
Dropout	Regularization

Weight decay	L2 regularization
Scheduler	Learning-rate dynamics

Önce LR
Deep learning tuningde her şeyi aynı anda aramak yerine learning rate ve optimization stability ile başlamak daha verimlidir.

33. Data leakage during tuning

Leak	Örnek
Preprocessing leakage	Scaler tüm dataya fit
Target encoding leakage	Validation target encoding map'e girdi
Feature selection leakage	Tüm datada feature seçildi
SMOTE leakage	Split öncesi oversampling
Time leakage	Random CV ile future data train'e girdi

Çözüm
Bütün learnable preprocessing adımlarını CV-aware pipeline içinde tut.

34. Search artifacts ve experiment tracking

- Trial parameters.
- Metric values.
- CV mean/std.
- Random seed ve dataset version.
- Runtime / hardware.
- Code / git commit.

MLOps bağlantısı
Tuning sonucunu sadece notebook çıktısı olarak bırakma; tekrar üretilebilir experiment kaydı haline getir.

35. Gerçek vaka: churn tuning

Retention modeli için positive class %8 ve ekip haftalık 20,000 müşteriye ulaşabiliyor.

Adım	Karar
Metric	PR-AUC primary; precision@20k secondary
Split	Temporal CV
Model	LightGBM / CatBoost benchmark
Search	Random/Optuna 60-100 trial
Params	lr, leaves/depth, min_leaf, subsample, regularization
Early stopping	Her fold içinde
Final policy	Selected model + capacity threshold

Başarı

Best AUC değil, temporal foldlarda stabil PR-AUC ve top-k değer üreten config aranır.

36. Gerçek vaka: house price regression

Karar	Örnek
Metric	MAE primary, RMSE guardrail
CV	Time-aware / geography-aware gerekebilir
Models	Ridge, Random Forest, boosting
Search	Log alpha veya random boosting space
Segment	Luxury segment MAE ayrıca raporlanır

Model seçimi
0.5% daha iyi global MAE ama lüks segmentte ciddi bias varsa business olarak tercih edilmeyebilir.

37. Gerçek vaka: fraud tuning

- Positive prevalence çok düşük.
- PR-AUC ve precision@review-capacity kullan.
- scale_pos_weight/class weights search edilebilir.
- Temporal split zorunluya yakındır.
- Threshold final policy aşamasında seçilir.

Compute vs latency
En iyi offline model production latency limitini aşarsa search objective'e inference cost/latency constraint eklemek gerekebilir.

38. Tuning anti-patternleri

- Test set üzerinde hyperparameter seçmek.
- Default CV ile time/group structure bozmak.
- Learning rate için lineer aralık kullanmak.
- Search space'i anlamsız derecede büyük açmak.
- Best single score dışında trial distributiona bakmamak.
- Threshold ve model tuningini aynı şey sanmak.
- Search sonrası manuel validation hacking yapmak.
- Random seed ve experiment metadata kaydetmemek.
- Pipeline dışı preprocessing ile leakage yaratmak.
- Compute budgeti business value ile karşılaştırmamak.

39. Tuning checklist

- Primary metric önceden seçildi mi?
- CV deployment gerçeğini taklit ediyor mu?

- Search space domain bilgisiyle sınırlandı mı?
- Log-scale gereken parametreler doğru dağılımda mı?
- Trial budget hesaplandı mı?
- Early stopping/pruning var mı?
- Preprocessing pipeline içinde mi?
- Validation overfitting riski değerlendirildi mi?
- Top configurations/stability incelendi mi?
- Test set final aşamaya kadar saklandı mı?

40. Mülakat soruları

Soru	Kısa cevap
Grid Search ile Random Search farkı?	Grid tüm sabit kombinasyonları, random dağılımdan sınırlı sayıda örneği dener.
Random Search ne zaman iyi?	Büyük/high-dimensional search space ve sınırlı budget.
Neden loguniform?	Order-of-magnitude duyarlı parametreleri daha doğal örneklemek için.
Nested CV neden?	Tuning biasından bağımsız generalization estimate için.
Early stopping nedir?	Validation iyileşmesi durunca iterative trainingi kesmek.
Pruning nedir?	Umut vermeyen hyperparameter trialını erken sonlandırmak.
Optuna ne sağlar?	Programatik search space, sampler, pruning ve study tracking.
Threshold tuning hyperparameter tuning midir?	Ayrı karar katmanıdır; model skorunu aksiyona çevirir.
Validation overfitting nasıl olur?	Çok trial validation noiseuna uyum sağlayabilir.
Time series tuningde ne değişir?	Random CV yerine temporal folds kullanılır.
Best score neden yeterli değil?	CV variance ve çevredeki stability de önemlidir.
Search budget nasıl hesaplanır?	Trials x folds x fit maliyeti.
Multi-metric GridSearchCV nasıl seçer?	refit ile primary metric belirlenir.
Search space neden önemlidir?	Optimizer yalnız tanımlanan bölgede arama yapabilir.
Tuning leakage nasıl önlenir?	Learnable preprocessing CV-aware pipeline içinde fit edilir.

41. Uygulama alıştırmaları

10. Ridge alpha için linear grid ve logspace grid karşılaştır.
11. Random Forest için Grid vs Random Search performans/fit sayısı kıyasla.
12. Imbalanced classificationda ROC-AUC ile PR-AUC tuning sonucunu karşılaştır.
13. TimeSeriesSplit ve random KFold ile best params farkını incele.
14. Optuna ile 50-trial study kur ve optimization history çiz.
15. Early stopping açık/kapalı boosting runtime ve score karşılaştır.
16. Top 10 trialın CV std değerlerini raporla.
17. Trial budgeti 20/50/100 yapıp diminishing returns gözlemle.
18. Nested CV ile normal CV performance estimate farkını ölç.
19. Custom business cost metric ile threshold sonrası model seçimi yap.

42. Mini portföy projesi

Proje: Hyperparameter Optimization Benchmark - Churn Risk

Çıktı	İçerik

01_baseline.ipynb	Default model benchmark
02_random_search.ipynb	Broad search
03_optuna.ipynb	Bayesian/TPE study + pruning
04_nested_cv.ipynb	Unbiased performance estimate
05_threshold.ipynb	Capacity/cost operating point
trials.csv	Trial params, scores, runtime
report.md	Search strategy, stability, final decision

CV cümlesi
Built a leakage-safe hyperparameter optimization framework comparing randomized search and Optuna with time-aware cross-validation, pruning, nested evaluation and business-oriented operating-point selection.

43. Final cheat sheet

Kavram	Hatırlanacak fikir
Grid Search	Küçük/discrete space
Random Search	Büyük space, fixed budget
Optuna	Adaptive study + pruning
Search space	Optimization kalitesinin temeli
Log scale	LR/regularization gibi order-of-magnitude parametreler
CV	Config performance tahmini
Nested CV	Tuningden bağımsız evaluation
Early stopping	Tek trainingi erken kes
Pruning	Kötü trialı erken kes
Validation overfit	Çok trial validation noiseuna uyabilir
Multi-metric	Primary metrici refit ile belirle
Threshold	Model tuning sonrası operating point
Tracking	Trial + data + code + runtime kaydı
Budget	Trials x folds x fit cost
Stability	Best score kadar önemlidir

Bu modülden sonra
Imbalanced Data ve Explainable AI modüllerinde modelin yalnız skorunu değil; karar eşiğini, class weightingini ve yorumlanabilirliğini de sistematik olarak optimize edebileceksin.